

Inigo Quilez :: articles :: cppencapsulation - 2012



Intro

So here's a little coding trick I've developed in 2012 to workaround the lack of encapsulation in the C++ language. I've edited the article with some more modern constructs because I still find the technique useful, and I hope others do too.

As we know, C++ classes don't provide much encapsulation - many implementation details leak to the user. Consider this class:

number_bucket.h

```
#pragma once

#include <vector>
#include <random>

class NumberBucket
{
public:
    NumberBucket();
    ~NumberBucket();

    void addNumber( int n );
    int pickNumber( void );

private:
    std::vector<int> mNumbers;
    std::minstd_rand0 mRandoms;
};
```

number_buket.cpp

```

#include <cassert>
#include <vector>
#include <random>
#include "number_bucket.h"

NumberBucket::NumberBucket():mNumbers(),mRandoms(12345) {}
NumberBucket::~NumberBucket() {}

void NumberBucket::addNumber( int n )
{
    mNumbers.push_back(n);
}

int NumberBucket::pickNumber( void )
{
    const size_t size = mNumbers.size();
    assert( size>=1 );
    const size_t id = mRandoms() % size;
    return mNumbers[ id ];
}

```

Side notes - [1] this is not how you pick uniformly from a set of integers. [2] I'd never write a class with constructors, I'm writing traditional C++ here only to keep the conversation focused on encapsulation.

There are a few problems with this class. First, many implementation details and data types are leaking. In particular code that uses **NumberBucket** needs to pull `<vector>` and `<random>` even though interacting with this class' API does not require any of those data types. The involvement of vectors and random number generators are internal implementation details of the class and should not have ever concerned the caller.

To make things worse, code that uses **NumberBucket** is also forced its own client code to include `<vector>` and `<random>`. This dependency chain propagates quickly into a cascade of include files, where data types leak upstream all the way to the root of the project, making build times explode to 5 minutes for projects that with manual encapsulation and basic include hygiene actually only take 5 seconds to build. This is specially bad for `std::` library, which is very leaky. Forward-declaring classes can mitigate some issues, at the cost of introducing indirections, but is still not proper encapsulation.

The problem of encapsulation is larger than just data type and include file leakage though. Imagine **NumberBucket** used a plain `mNumbers[]` array instead of a `std::vector` and some inlined `mRandom` generator instead of `std::minstd_rand0`. The class would still reveal implementation details, such as the existence of `mNumbers` and `mRandom`. This adds cognitive load when inspecting and exploring this class' interface to learn how to interact with it. And also triggers recompilation of all upstream code when modifying the private members of the class, even though they do not affect the class API at all.

The traditional solution - PIMPL

The most common way to fight the lack of encapsulation in C++ is the PIMPL pattern ("Pointer to IMPLementation"). PIMPL hides all implementation details from the class' interface, including data types. This is an example of PIMPL applied to the NumberBucket class above:

number_bucket.h

```
#pragma once

#include <memory>

class NumberBucket
{
public:
    NumberBucket();
    ~NumberBucket();

    void addNumber( int n );
    int pickNumber( void );

private:
    struct Impl;
    std::unique_ptr<Impl> mImpl;
};
```

number_bucket.cpp

```
#include <cassert>
#include <vector>
#include <random>
#include "number_bucket.h"

struct NumberBucket::Impl
{
    Impl():mNumbers(),mRandoms(12345) {}
    ~Impl() {}
```

```

        std::vector<int> mNumbers;
        std::minstd_rand0 mRandoms;
    };

    NumberBucket::NumberBucket() : mImpl(std::make_unique()) {}
    NumberBucket::~NumberBucket() {}

    void NumberBucket::addNumber( int n )
    {
        mImpl->mNumbers.push_back(n);
    }

    int NumberBucket::pickNumber( void )
    {
        const size_t size = mImpl->mNumbers.size();
        assert( size>=1 );
        const size_t id = mImpl->mRandoms() % size;
        return mImpl->mNumbers[ id ];
    }

```

This works well and I used it a lot in the 2000s (with **void *** instead of **unique_ptr**). However it has one undesirable feature - it requires an extra heap allocation at initialization time, and an indirection for every object operation. The heap allocation happens at **unique_ptr** construction time. If you want an even cleaner API where `<memory>` header is not leaked all over the place, you can use a raw pointer and `new()/delete()`. This also has the benefit that you can return errors on initialization instead of throwing from constructors ala RAII, but your philosophy here might vary.

In my case, the reason I stopped using this vanilla variant of PIMPL is that the heap allocation creates memory fragmentation. The actual meat of the implementation operates in a memory location that is far from where the actual class sits. This can create performance problems in many situations, which of course can be solved by introducing a memory pool that manages all IMPLementations instances of the same type. But at that point the amount of work and boilerplate required to achieve performant encapsulation starts to feel ridiculous. So here's where I landed in a different implementation of the PIMPL pattern:

The modified solution - In-Place PIMPL

So let's finally explore the solution I landed on years ago: just use PIMPL but instead of allocating space in the heap for our private implementation, do it in the class itself in an opaque block of memory:

number_bucket.h

```
#pragma once

class NumberBucket
{
public:
    NumberBucket();
    ~NumberBucket();

    void addNumber( int n );
    int pickNumber( void );

private:
    int opaque[8];
};
```

number_bucket.cpp

```
#include <cassert>
#include <vector>
#include <random>
#include <new>
#include "number_bucket.h"

struct Impl
{
    Impl():mNumbers(),mGenerator(12345) {}
    ~Impl() {}

    std::vector<int> mNumbers;
    std::minstd_rand0 mGenerator;
};

#define mImpl ((Impl*)opaque)

NumberBucket::NumberBucket() { new(opaque)Impl(); }
NumberBucket::~~NumberBucket() { mImpl->~Impl(); }

void NumberBucket::addNumber( int n )
{
    static_assert( sizeof(opaque)>=sizeof(Impl) );
    mImpl->mNumbers.push_back(n);
}
```

```
int NumberBucket::pickNumber( void )
{
    const size_t size = mImpl->mNumbers.size();
    assert( size >= 1 );
    const size_t id = mImpl->mGenerator() % size;
    return mImpl->mNumbers[ id ];
}
```

As you can see, the member "`int opaque[8]`" in where we instantiate our private implementation class. We do so by constructing/initializing it in place (with placement new or otherwise). Internally, all the accesses to the private implementation require casting the opaque blob to the implementation class, but this does not result in an indirection at runtime. Instead this is just accessing members as a fixed offset known at compile time, just like in regular classes. I like using `int[size/4]` instead of `char[size]` for the opaque blob so I ensure proper memory alignment.

Obviously the size of the opaque blob needs to be large enough to host the implementation class. We make sure that's the case with the `static_assert` which compares the size of the blob and the size of the implementation class. I often check for equality to prevent wasting space, unless the class needs different implementations in different platforms.

Lastly, there are no include files required, and as a result files only using **NumberBucket** and classes written in the same idiom can now be compiled at the speed of light.

But now let's address the major drawback of this strategy - the size of the opaque block has to be maintained by hand in the class definition. You might call this "a hack", but for me this has become more an idiom, and the pros of this In-Place PIMPLE approach largely outweigh the cons: it achieves true encapsulation (no include explosion, no implementation detail leaks) without performance impact (no heaps, no indirections), and with less code and complexity than PIMPL (no constructors and factories required).

Obviously I'd like C++ to provide encapsulation and not have to do any of this, but I've been a fan of the technique since I developed it and I've used it in professional products and hobby projects alike. If you want to have a cleaner programs and shorter build times, give it a try!

inigo quilez - learning computer graphics since 1994